

Library Management System — Java OOP Practice

Project Objective

Build a console-based Library Management System using Java.

Practice goals:

- Classes & Objects
- Constructors
- Encapsulation, getters and setters
- Inheritance
- Polymorphism
- Abstraction
- ArrayList
- Loops, conditions and switch
- Methods and basic validation

1. Book Management

The system should allow the librarian to:

1. Add a new book
2. Display all books
3. Search for a book by title
4. Borrow a book
5. Return a book

Each book should contain:

- Book ID
- Title
- Author
- Availability status

Create a Book class with private attributes.

Define the required constructor, getters, setters, and methods.

2. User Management

The system should support different types of library users.

The general User type contains:

- User ID
- User Name

Supported users:

- Student
- Teacher

Class relationship:

User

■■■ Student

■■■ Teacher

Use inheritance, abstraction, and method overriding where appropriate.

3. User Roles

Each user type should display its own role.

Example:

Student → Role: Student

Teacher → Role: Teacher

Use method overriding to provide the appropriate role for each user type.

4. Library Class

Create a Library class responsible for managing the books.

The Library should contain a collection of books.

Required operations:

- addBook()
- displayBooks()
- searchBook()
- borrowBook()
- returnBook()

Keep the library business logic inside the Library class.

5. Main Application

Create a menu similar to:

===== Library Management System =====

1. Add Book
2. Display Books
3. Search Book
4. Borrow Book
5. Return Book
6. Display User Information
7. Exit

Enter Choice:

The menu should continue until Exit is selected.

6. Validation

Add validation for invalid data.

Examples:

- Book ID cannot be negative.
- Book title cannot be empty.
- User name cannot be empty.

Use encapsulation and conditions to protect object data.

7. Duplicate Books

Prevent duplicate Book IDs.

Example:

Enter Book ID: 101

Book added successfully.

Enter Book ID: 101

Error: Book ID already exists.

Check the Book ID before adding a book to the library.

8. Borrowing Rules

Implement borrowing rules.

A book that is already borrowed cannot be borrowed again.

Example:

First attempt:

Book Borrowed Successfully

Second attempt:

Book Already Borrowed

9. User-Specific Borrowing

Store who borrowed each book.

Instead of storing only the availability status, keep track of the user who borrowed the book.

Example:

Java Basics

Borrowed by: Ahmed

Relationship:

User → borrows → Book

10. Borrowing Limits

Add different borrowing limits for different user types.

Example:

- Student → maximum 3 books
- Teacher → maximum 5 books

Use polymorphism or an appropriate OOP design to represent the different rules.

11. Advanced OOP Challenge

Extend the system with another user type.

Example:

User

■■■ Student

■■■ Teacher

■■■ Librarian

Design the system so that adding a new user type requires minimal changes to existing classes.

12. Suggested Class Structure

Book

- bookId
- title
- author
- availability

User

- userId
- userName
- showRole()

Student

Teacher

Library

- books
- addBook()
- displayBooks()
- searchBook()
- borrowBook()
- returnBook()

Main

- menu
- user interaction

13. Assignment Levels

Level 1 — OOP Basics

Build the basic Library Management System.

Level 2 — Business Rules

Add validation, duplicate detection, borrowing limits, and user-specific borrowing.

Level 3 — OOP Challenge

Improve the design using abstraction and polymorphism and introduce another user type.

Focus on clean class design and clear responsibilities.